

Notebook: all_algo_comparsons/linear_regression_comparison.ipynb

[--- MARKDOWN CELL ---]

Linear Regression - Algorithm Comparison

This notebook implements Linear Regression from scratch and compares it with sklearn on housing price prediction.

[--- CODE CELL ---]

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression as SklearnLR
from sklearn.metrics import mean_squared_error, r2_score, mean_absolute_error
```

[--- CODE CELL ---]

```
np.random.seed(42)
```

[--- MARKDOWN CELL ---]

Linear Regression Implementation From Scratch

[--- CODE CELL ---]

```
class LinearRegressionScratch:
    """Linear Regression from scratch using gradient descent."""
    def __init__(self, lr=0.01, epochs=1000):
        self.lr = lr
        self.epochs = epochs
        self.w = None
        self.b = None
        self.cost_history = []
    def compute_cost(self, X, y):
        """Compute Mean Squared Error."""
        m = len(y)
        y_pred = np.dot(X, self.w) + self.b
        cost = (1/(2*m)) * np.sum((y_pred - y) ** 2)
        return cost
    def fit(self, X, y):
        """Fit model using gradient descent."""
        n_samples, n_features = X.shape
        self.w = np.zeros(n_features)
        self.b = 0
        self.cost_history = []
        for _ in range(self.epochs):
            y_pred = np.dot(X, self.w) + self.b
            error = y_pred - y
            # Gradients
            dw = (1/n_samples) * np.dot(X.T, error)
            db = (1/n_samples) * np.sum(error)
            # Update
            self.w -= self.lr * dw
            self.b -= self.lr * db
            cost = self.compute_cost(X, y)
            self.cost_history.append(cost)
        return self
```

```
def predict(self, X):
    """Predict values."""
    return np.dot(X, self.w) + self.b

def score(self, X, y):
    """Return R2 score."""
    y_pred = self.predict(X)
    return r2_score(y, y_pred)
```

[--- MARKDOWN CELL ---]

Load California Housing Dataset

[--- CODE CELL ---]

```
# Load California Housing dataset
housing = fetch_california_housing()
X, y = housing.data, housing.target
print(f"California Housing Dataset:")
print(f"  Samples: {X.shape[0]}, Features: {X.shape[1]}")
print(f"  Feature names: {housing.feature_names}")
```

[--- CODE CELL ---]

```
# Create DataFrame
df = pd.DataFrame(X, columns=housing.feature_names)
df['Target'] = y
df.head()
```

[--- CODE CELL ---]

```
df.describe()
```

[--- CODE CELL ---]

```
# Target distribution
plt.figure(figsize=(10, 5))
plt.hist(y, bins=50, edgecolor='black', alpha=0.7)
plt.xlabel('Median House Value (in $100,000s)')
plt.ylabel('Frequency')
plt.title('Target Distribution - California Housing Prices')
plt.grid(True, alpha=0.3)
plt.show()
```

[--- CODE CELL ---]

```
# Split and scale
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

[--- MARKDOWN CELL ---]

Train and Compare Models

[--- CODE CELL ---]

```
# Train From Scratch
lr_scratch = LinearRegressionScratch(lr=0.1, epochs=1000)
lr_scratch.fit(X_train_scaled, y_train)

# Train Sklearn
```

```
lr_sklearn = SklearnLR()
lr_sklearn.fit(X_train_scaled, y_train)

# Predictions
y_pred_scratch = lr_scratch.predict(X_test_scaled)
y_pred_sklearn = lr_sklearn.predict(X_test_scaled)
```

[--- CODE CELL ---]

```
# Evaluation metrics
metrics = {
    'Model': ['From Scratch', 'Sklearn'],
    'R2 Score': [
        r2_score(y_test, y_pred_scratch),
        r2_score(y_test, y_pred_sklearn)
    ],
    'MSE': [
        mean_squared_error(y_test, y_pred_scratch),
        mean_squared_error(y_test, y_pred_sklearn)
    ],
    'MAE': [
        mean_absolute_error(y_test, y_pred_scratch),
        mean_absolute_error(y_test, y_pred_sklearn)
    ]
}

metrics_df = pd.DataFrame(metrics)
print("Performance Comparison:")
print(metrics_df.to_string(index=False))
```

[--- CODE CELL ---]

```
# Plot training curve
plt.figure(figsize=(10, 5))
plt.plot(lr_scratch.cost_history)
plt.xlabel('Epochs')
plt.ylabel('Cost (MSE)')
plt.title('Training Loss Curve - Linear Regression From Scratch')
plt.grid(True, alpha=0.3)
plt.show()
```

[--- MARKDOWN CELL ---]

Visualization

[--- CODE CELL ---]

```
# Actual vs Predicted
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# From Scratch
axes[0].scatter(y_test, y_pred_scratch, alpha=0.5)
axes[0].plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--', lw=2)
axes[0].set_xlabel('Actual')
axes[0].set_ylabel('Predicted')
axes[0].set_title('From Scratch: Actual vs Predicted')
axes[0].grid(True, alpha=0.3)

# Sklearn
axes[1].scatter(y_test, y_pred_sklearn, alpha=0.5, color='orange')
axes[1].plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--', lw=2)
axes[1].set_xlabel('Actual')
axes[1].set_ylabel('Predicted')
axes[1].set_title('Sklearn: Actual vs Predicted')
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```

[--- CODE CELL ---]

```
# Residuals
residuals_scratch = y_test - y_pred_scratch
residuals_sklearn = y_test - y_pred_sklearn
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
axes[0].hist(residuals_scratch, bins=50, edgecolor='black', alpha=0.7)
axes[0].axvline(x=0, color='r', linestyle='--')
axes[0].set_xlabel('Residual')
axes[0].set_ylabel('Frequency')
axes[0].set_title('From Scratch: Residual Distribution')
axes[1].hist(residuals_sklearn, bins=50, edgecolor='black', alpha=0.7, color='orange')
axes[1].axvline(x=0, color='r', linestyle='--')
axes[1].set_xlabel('Residual')
axes[1].set_ylabel('Frequency')
axes[1].set_title('Sklearn: Residual Distribution')
plt.tight_layout()
plt.show()
```

[--- CODE CELL ---]

```
# Comparison bar chart
fig, axes = plt.subplots(1, 3, figsize=(15, 5))
metrics_to_plot = ['R2 Score', 'MSE', 'MAE']
colors = ['steelblue', 'coral']
for i, metric in enumerate(metrics_to_plot):
    values = metrics_df[metric].values
    bars = axes[i].bar(['From Scratch', 'Sklearn'], values, color=colors, edgecolor='black')
    axes[i].set_title(metric)
    axes[i].grid(True, alpha=0.3, axis='y')
    for bar in bars:
        axes[i].text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.01,
                     f'{bar.get_height():.4f}', ha='center', va='bottom')
plt.suptitle('Linear Regression: From Scratch vs Sklearn', fontsize=14, y=1.02)
plt.tight_layout()
plt.show()
```

[--- MARKDOWN CELL ---]**## Conclusion**

The Linear Regression implementation from scratch achieves comparable performance to sklearn:

- R^2 scores are very similar between both implementations
- The gradient descent approach converges to a similar solution as the closed-form solution
- Residual distributions are nearly identical, confirming equivalent model quality